# Automation Script to fire Bulk APIs to Director Templates /Devices

| Author | Arun Chandar |
|---|---|
| | |

### Purpose

This article will help to fire Bulk APIs to Director at one go with minimalistic effort.

### Requisites:

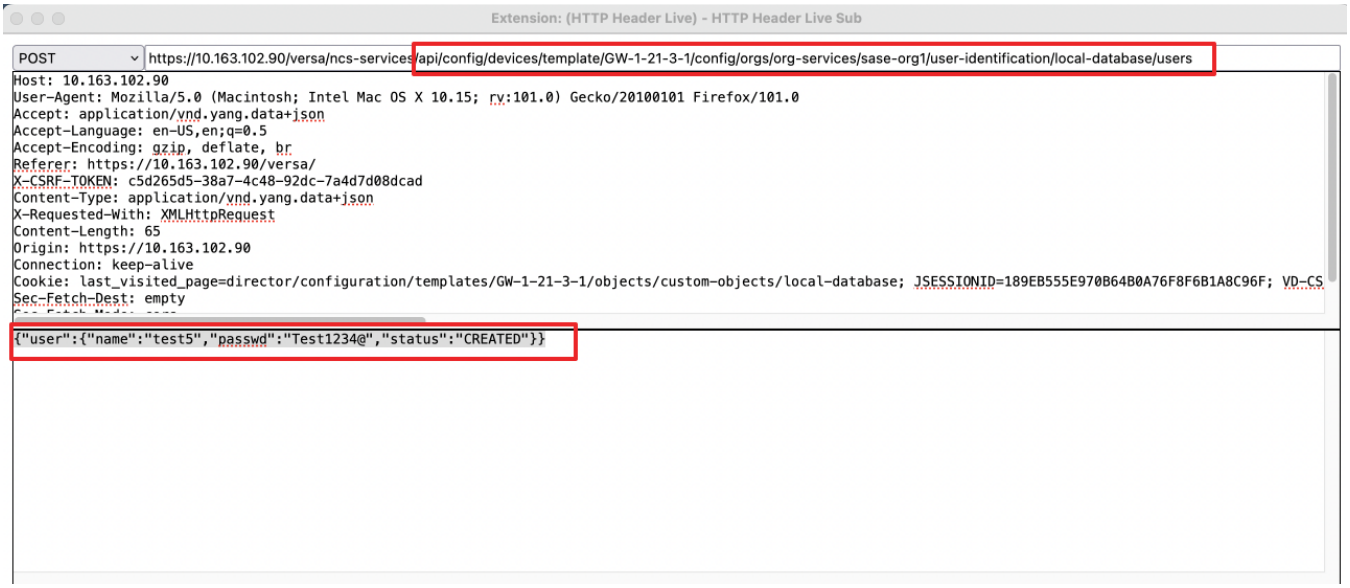
The following requisites should be met:

1. Python installed
2. Python subprocess module installed
3. curl installed
4. Director reachable over port 9182
5. Ubuntu Machine/Windows

### Working:

Lets take an example to import a list of local user database information on to the Template. To achieve this, we need to identify the right API data that would be used while creating users on the Director. For example, when the POST request gets triggered, the following data would be triggered for the local database creation:

| API Data |
|---|
| {"user":{"name":"test5","passwd":"Test1234@","status":"CREATED"}} |




To automate with multiple bulk requests use the attached csv file userdatabase-conversion.csv. Fill in the required user name in column - B and Password in column F as below:

| | A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|---|
| 1 | '{"user":{"name":" | tester1 | " | , | "passwd":" | Test223421 | " | , |
| 2 | '{"user":{"name":" | tester2 | " | , | "passwd":" | Test223422 | " | , |
| 3 | '{"user":{"name":" | tester3 usernames | " | , | "passwd":" | Test223423 | " | , |
| 4 | '{"user":{"name":" | tester4 | " | , | "passwd":" | Test223424 | " | , |
| 5 | '{"user":{"name":" | tester5 | " | , | "passwd":" | Test223425 | " | , |
| 6 | '{"user":{"name":" | tester6 | " | , | "passwd":" | Test223426 | " | , |
| 7 | '{"user":{"name":" | tester7 | " | , | "passwd":" | Test223427 | " | , |
| 8 | '{"user":{"name":" | tester8 | " | , | "passwd":" | Test223428 | " | , |
| 9 | '{"user":{"name":" | tester9 | " | , | "passwd":" | Test223429 | " | , |
| 10 | '{"user":{"name":" | tester10 | " | , | "passwd":" | Test223430 | " | , |
| 11 | '{"user":{"name":" | tester11 | " | , | "passwd":" | Test223431 | " | , |
| 12 | '{"user":{"name":" | tester12 | " | , | "passwd":" | Test223432 | " | , |
| 13 | '{"user":{"name":" | tester13 | " | , | "passwd":" | Test223433 | " | , |
| 14 | | | | | | | | |
| 15 | | | | | | | | |
| 16 | | | | | | | | |
| 17 | | | | | | | | |

Once the required data is created, copy paste all the contents of the csv file to a text file:

| | A | B | C | D | E | F | G | |
|---|---|---|---|---|---|---|---|---|
| 1 | '{"user":{"name":" | tester1 | " | , | "passwd":" | Test223421 | " | , |
| 2 | '{"user":{"name":" | tester2 | " | , | "passwd":" | Test223422 | " | , |
| 3 | '{"user":{"name":" | tester3 | " | , | "passwd":" | Test223423 | " | , |
| 4 | '{"user":{"name":" | tester4 | " | , | "passwd":" | Test223424 | " | , |
| 5 | '{"user":{"name":" | tester5 | " | , | "passwd":" | Test223425 | " | , |
| 6 | '{"user":{"name":" | tester6 | " | , | "passwd":" | Test223426 | " | , |
| 7 | '{"user":{"name":" | tester7 | " | , | "passwd":" | Test223427 | " | , |
| 8 | '{"user":{"name":" | tester8 | " | , | "passwd":" | Test223428 | " | , |
| 9 | '{"user":{"name":" | tester9 | " | , | "passwd":" | Test223429 | " | , |
| 10 | '{"user":{"name":" | tester10 | " | , | "passwd":" | Test223430 | " | , |
| 11 | '{"user":{"name":" | tester11 | " | , | "passwd":" | Test223431 | " | , |
| 12 | '{"user":{"name":" | tester12 | " | , | "passwd":" | Test223432 | " | , |
| 13 | '{"user":{"name":" | tester13 | " | , | "passwd":" | Test223433 | " | , |
| 14 | | | | | | | | |
| 15 | | | | | | | | |
| 16 | | | | | | | | |
| 17 | | | | | | | | |
| 18 | | | | | | | | |

```
lab@root:~$ cat userdatabase-conv
cat: userdatabase-conv: No such file or directory
lab@root:~$ cat userdatabase-conv.txt
'{"user":{"name":"      tester1 "      ,      "passwd":"      Test223421      " ,      "status":"CREATED"}}'
'{"user":{"name":"      tester2 "      ,      "passwd":"      Test223422      " ,      "status":"CREATED"}}'
'{"user":{"name":"      tester3 "      ,      "passwd":"      Test223423      " ,      "status":"CREATED"}}'
'{"user":{"name":"      tester4 "      ,      "passwd":"      Test223424      " ,      "status":"CREATED"}}'
'{"user":{"name":"      tester5 "      ,      "passwd":"      Test223425      " ,      "status":"CREATED"}}'
'{"user":{"name":"      tester6 "      ,      "passwd":"      Test223426      " ,      "status":"CREATED"}}'
'{"user":{"name":"      tester7 "      ,      "passwd":"      Test223427      " ,      "status":"CREATED"}}'
'{"user":{"name":"      tester8 "      ,      "passwd":"      Test223428      " ,      "status":"CREATED"}}'
'{"user":{"name":"      tester9 "      ,      "passwd":"      Test223429      " ,      "status":"CREATED"}}'
'{"user":{"name":"      tester10       "      ,      "passwd":"      Test223430      " ,      "status":"CREATED"}}'
'{"user":{"name":"      tester11       "      ,      "passwd":"      Test223431      " ,      "status":"CREATED"}}'
'{"user":{"name":"      tester12       "      ,      "passwd":"      Test223432      " ,      "status":"CREATED"}}'
'{"user":{"name":"      tester13       "      ,      "passwd":"      Test223433      " ,      "status":"CREATED"}}'
lab@root:~$
```

Save the file, the rest of the formatting and firing of APIs will be taken care by the Script.

Next, download the script bulk-api-automation.py. On Ubuntu or Windows place the txt file and the script (.py) file on to the same directory:

```
lab@root:~/Automation$ cd ..
lab@root:~$ cp bulk-api-automation.py Automation/
lab@root:~$ cp userdatabase-conv.txt Automation/
lab@root:~$ cd Automation/
lab@root:~/Automation$ ls
bulk-api-automation.py  userdatabase-conv.txt  Zone-DoS-Protection
```

Change the credentials of your Director in the script as below:

```
import subprocess
# read in users and strip the newlines
with open(raw_input("Enter API Filename: "),'r+') as f:
        lines = f.readlines()
        lines = [line.lstrip() for line in lines]
        lines = [line.replace("\t", "") for line in lines]
        d1 = [line.rstrip() for line in lines]
        print(d1)

data2 = raw_input("Enter API URL: ")

api = []
for data in d1:
    api.append('curl -k -d {} --user Administrator:Versa1234@ -X POST {} --insecure -H "Content-Type:application/json" -w "HTTP- %{{http_code}} code\n"'.format(data,data2))
print (api)
results=[]

for apis in api:
    results.append(subprocess.call(apis, shell=True))
    print(apis)

#for i,result in enumerate(results):
#    if result == 0:
#        print(apis[i])
```

Now, we can run the script. When executed, the script will ask for the API data filename. This filename is the file that actually contents the data to be created when the APIs gets triggered as below.

```
lab@root:~/Automation$ python bulk-api-automation.py
Enter API Filename: userdatabase-conv.txt
```

Once the file name is triggered, the script will sanitise and format into the correct format which can then be used as data feeds while triggering post requests to the director as below:

```
lab@root:~/Automation$ python bulk-api-automation.py
Enter API Filename: userdatabase-conv.txt
['\'{"user":{"name":"tester1","passwd":"Test223421","status":"CREATED"}}\'', '\'{"user":{"name":"tester2","passwd":"Test223422","status":"CREATED"}}\'', '\'{"user":{"name"
tester3","passwd":"Test223423","status":"CREATED"}}\'', '\'{"user":{"name":"tester4","passwd":"Test223424","status":"CREATED"}}\'', '\'{"user":{"name":"tester5","passwd":"
st223425","status":"CREATED"}}\'', '\'{"user":{"name":"tester6","passwd":"Test223426","status":"CREATED"}}\'', '\'{"user":{"name":"tester7","passwd":"Test223427","status":
REATED"}}\'', '\'{"user":{"name":"tester8","passwd":"Test223428","status":"CREATED"}}\'', '\'{"user":{"name":"tester9","passwd":"Test223429","status":"CREATED"}}\'', '\'{"
er":{"name":"tester10","passwd":"Test223430","status":"CREATED"}}\'', '\'{"user":{"name":"tester11","passwd":"Test223431","status":"CREATED"}}\'', '\'{"user":{"name":"test
12","passwd":"Test223432","status":"CREATED"}}\'', '\'{"user":{"name":"tester13","passwd":"Test223433","status":"CREATED"}}\'']
```

Next input the API URL associated to your director. For example, the below url would be used to create local user-database on to a particular template:

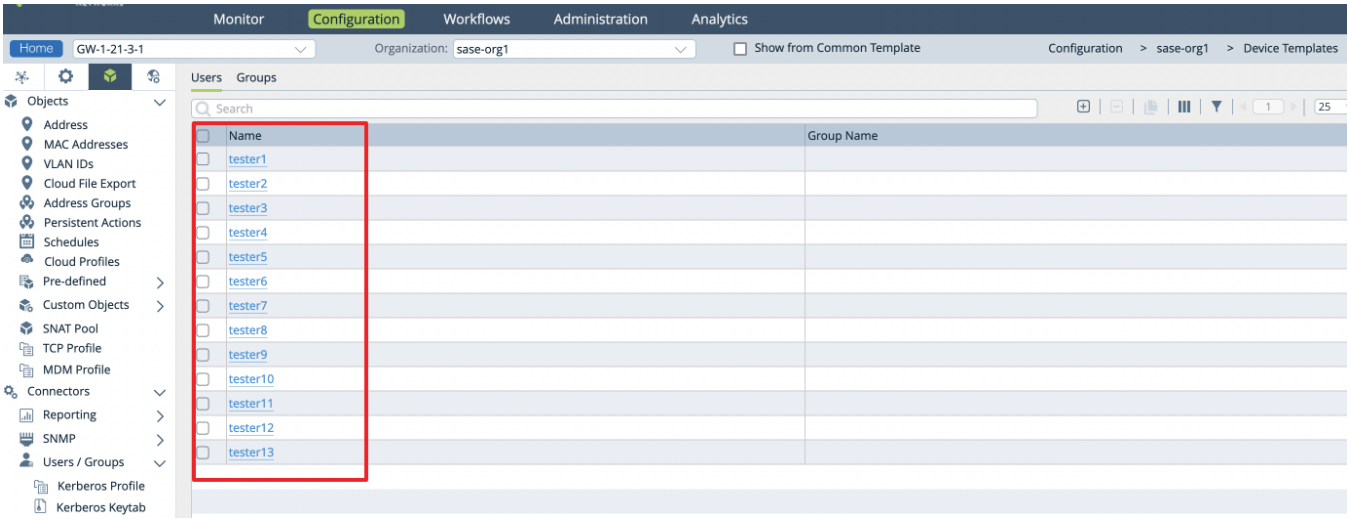
| API URL for Local User Database |
| --- |
| https://<<Director-IP>>:9182/api/config/devices/template/<<Template-Name>/config/orgs/org-services/<<ORG-NAME>>/user-identification/local-database/users |

```
lab@root:~/Automation$ python bulk-api-automation.py
Enter API Filename: userdatabase-conv.txt
['\'{"user":{"name":"tester1","passwd":"Test223421","status":"CREATED"}}\'', '\'{"user":{"name":"tester2","passwd":"Test223422","status":"CREATED"}}\'', '\'{"user":{"name":"
tester3","passwd":"Test223423","status":"CREATED"}}\'', '\'{"user":{"name":"tester4","passwd":"Test223424","status":"CREATED"}}\'', '\'{"user":{"name":"tester5","passwd":"Te
st223425","status":"CREATED"}}\'', '\'{"user":{"name":"tester6","passwd":"Test223426","status":"CREATED"}}\'', '\'{"user":{"name":"tester7","passwd":"Test223427","status":"C
REATED"}}\'', '\'{"user":{"name":"tester8","passwd":"Test223428","status":"CREATED"}}\'', '\'{"user":{"name":"tester9","passwd":"Test223429","status":"CREATED"}}\'', '\'{"us
er":{"name":"tester10","passwd":"Test223430","status":"CREATED"}}\'', '\'{"user":{"name":"tester11","passwd":"Test223431","status":"CREATED"}}\'', '\'{"user":{"name":"tester
12","passwd":"Test223432","status":"CREATED"}}\'', '\'{"user":{"name":"tester13","passwd":"Test223433","status":"CREATED"}}\'']
Enter API URL:  https://10.163.102.90:9182/api/config/devices/template/GW-1-21-3-1/config/orgs/org-services/sase-org1/user-identification/local-database/users
Secure Shell   Xterm   Zmodem   ZOC2206_Arun-Lab-219.201.log   01:38:11   173x26
```

Press enter and the script will start to construct the API requests based on the no.of data inputs available on the txt file and the script will start firing Bulk APIs based on the inputs given. The HTTP- 201 code indicates that the user has been successfully created on the Director:

```
HTTP- 201 code
curl -k -d '{"user":{"name":"tester8","passwd":"Test223428","status":"CREATED"}}' --user Administrator:Versa1234@ -X POST  https://10.163.102.90:9182/api/config/devices/temp
late/GW-1-21-3-1/config/orgs/org-services/sase-org1/user-identification/local-database/users --insecure -H "Content-Type:application/json" -w "HTTP- %{http_code} code
"
HTTP- 201 code
curl -k -d '{"user":{"name":"tester9","passwd":"Test223429","status":"CREATED"}}' --user Administrator:Versa1234@ -X POST  https://10.163.102.90:9182/api/config/devices/temp
late/GW-1-21-3-1/config/orgs/org-services/sase-org1/user-identification/local-database/users --insecure -H "Content-Type:application/json" -w "HTTP- %{http_code} code
"
HTTP- 201 code
curl -k -d '{"user":{"name":"tester10","passwd":"Test223430","status":"CREATED"}}' --user Administrator:Versa1234@ -X POST  https://10.163.102.90:9182/api/config/devices/ter
plate/GW-1-21-3-1/config/orgs/org-services/sase-org1/user-identification/local-database/users --insecure -H "Content-Type:application/json" -w "HTTP- %{http_code} code
"
HTTP- 201 code
curl -k -d '{"user":{"name":"tester11","passwd":"Test223431","status":"CREATED"}}' --user Administrator:Versa1234@ -X POST  https://10.163.102.90:9182/api/config/devices/ter
plate/GW-1-21-3-1/config/orgs/org-services/sase-org1/user-identification/local-database/users --insecure -H "Content-Type:application/json" -w "HTTP- %{http_code} code
"
HTTP- 201 code
curl -k -d '{"user":{"name":"tester12","passwd":"Test223432","status":"CREATED"}}' --user Administrator:Versa1234@ -X POST  https://10.163.102.90:9182/api/config/devices/ter
plate/GW-1-21-3-1/config/orgs/org-services/sase-org1/user-identification/local-database/users --insecure -H "Content-Type:application/json" -w "HTTP- %{http_code} code
"
HTTP- 201 code
curl -k -d '{"user":{"name":"tester13","passwd":"Test223433","status":"CREATED"}}' --user Administrator:Versa1234@ -X POST  https://10.163.102.90:9182/api/config/devices/ter
plate/GW-1-21-3-1/config/orgs/org-services/sase-org1/user-identification/local-database/users --insecure -H "Content-Type:application/json" -w "HTTP- %{http_code} code
```

Now, we can check the same in the Director to verify if all the users has been successfully created on the respective templates:




The same logic can be applied for other use cases as well with minimal effort spent on identifying the correct APIs to construct the csv file and then using it as a txt file.  This approach has been successfully used while creating multiple QoS configurations, creating huge list of services, Addresses, Firewall rules etc in SCB & AMP deployments.